

INSTRUCTION: Please answer all questions in the answer booklet. Show all your workings.

1. **[20 MARKS]** There are 5 processes (in order) of A (115 KB), B (500 KB), C (350 KB), D (200 KB), and E (450 KB) need to be placed in the memory.
- a) Suppose a fixed-partition is used with six memory partitions of 300 KB, 600 KB, 400 KB, 200 KB, 750 KB, and 125 KB (in order).
- (i) How would the best-fit and worst-fit algorithms place the processes? Draw the memory stack and label the partition size clearly. [10m]

Answer :

Fixed-partition: Exercises 8.3 (page: 384)

	Best-fit			Worst-fit		
Jobs: <i>A</i> (115 KB) <i>B</i> (500 KB) <i>C</i> (350 KB) <i>D</i> (200 KB) <i>E</i> (450 KB)	Main Memory			Main Memory		
	<i>Partition 0</i>	<i>OS</i>		<i>Partition 0</i>	<i>OS</i>	
	<i>Partition 1</i>		300KB	<i>Partition 1</i>	<i>D</i>	300KB
	<i>Partition 2</i>	<i>B</i>	600KB	<i>Partition 2</i>	<i>B</i>	600KB
	<i>Partition 3</i>	<i>C</i>	400KB	<i>Partition 3</i>	<i>C</i>	400KB
	<i>Partition 4</i>	<i>D</i>	200KB	<i>Partition 4</i>		200KB
	<i>Partition 5</i>	<i>E</i>	750KB	<i>Partition 5</i>	<i>A</i>	750KB
	<i>Partition 6</i>	<i>A</i>	125KB	<i>Partition 6</i>		125KB

- (ii) In terms of process placement in the memory, which algorithm faced a problem?
What is the problem and state the cause of the problem. [4m]

Worst-fit.

Problem: Process E cannot be placed in the memory for Worst-fit

Cause: since there is no enough available partition.

- b) **[10 MARKS]** Suppose a variable or dynamic partition is used with the same 5 processes in Question 1. The following events happened:
- t₁: process A and D left.
 - t₂: process F (50K) enters
 - t₃: process B left.
 - t₄: process G (550K) enters.

Draw the memory stack at t_2 , t_3 and t_4 and label the sizes clearly for all processes. Use first-fit algorithm and relocatable dynamic partitions of merging adjacent holes to place the process in the memory. [6m]

F	50
	65
B	500
C	350
	200
E	450

T2

F	50
	565
C	350
	200
E	450

T3

F	50
G	550
	15
C	350
	200
E	450

T4

2. [10 MARKS] Answer the following questions.

a) Consider a logical address space of 2^8 pages with a 4 KB page size, mapped onto a physical memory of 4096 frames. Its page table is given in Table 2.

Table 2

0	50
1	30
2	400
3	100

(i) What is the size of logical and physical addresses (in 2^n bytes)? [4m]

page size $\rightarrow 4 \text{ KB} = 2^{12}$

- [2m] Logical address = totalpage x page size = $2^8 \times 2^{12} = 2^{20}$ bytes
- [2m] Physical address = totalframe x frame size = (4096 pages) x (frame size = page size) = $2^{12} \times 2^{12} = 2^{24}$ bytes

(ii) Calculate the physical address if the logical address is 3 : 400. [3m]

3 $\rightarrow$ 100. Frame size = page size

$$\begin{aligned}
 \text{Physical address} &= (\text{frame\#} \times \text{frame size}) + \text{offset} \\
 &= (100 \times 2^{12}) + 400 \\
 &= (100 \times 4096) + 400 \\
 &= 4096000 + 400 \\
 &= 4096400
 \end{aligned}$$

(iii) What is the page number and offset for the logical address of 51152? [3m]

$$51152/4096 = 12.48828125$$

$$\text{Page Number: } 12 - 1 = 11$$

$$\text{Offset} = (12.48828125 - 12) \times 4096 = 2000 - 1 = 1999$$

3. [12 MARKS] Define the following terms.

a) Demand paging [3m]

Load pages only needed in virtual memory

b) Lazy swapper

Used in demand paging. Swaps a page into memory only when needed.

c) Belady's anomaly [3m]

Page fault rate may increase as the number of allocated frames increases, even though it looks like it should be the opposite effect.

d) Thrashing in virtual memory [3m]

High paging activity. Spending more time paging than executing.

4. [13 MARKS] Consider the following page reference string: 7, 2, 3, 1, 2, 5, 3, 4, 6, 7, 1, 3. Assuming demand paging with four frames, how many page faults would occur for the following replacement algorithms? Please circle which references cause page faults.

a) Optimal (OPT) replacement [6.5m]

7	2	3	1	2	5	3	4	6	7	1	3
7	7	7	7		7		7	7			
	2	2	2		5		4	6			
		3	3		3		3	3			
			1		1		1	1			

7 page fault

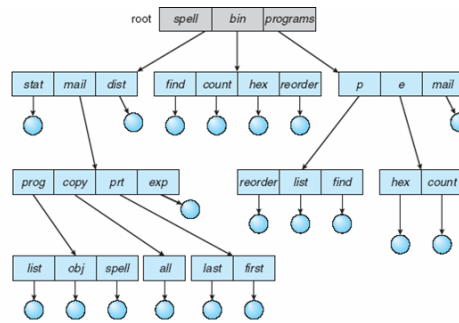
b) LRU replacement [6.5m]

7	2	3	1	2	5	3	4	6	7	1	3
7	7	7	7		5		5	5	7	7	7
	2	2	2		2		2	6	6	6	6
		3	3		3		3	3	3	1	1
			1		1		4	4	4	4	3

10 page fault

5. [10 MARKS] Draw and discuss the advantages of:

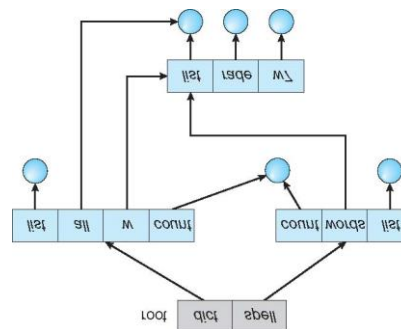
a) Tree-structured directories [5m]



Text book 10.3.5, slide 13.21- 13.24

- Extension of two-level directory, tree can have arbitrary height
- Separate directory for each user – they can create their own subdirectory and organize their file accordingly.
- The tree has a root directory and every file has unique path name,
- Can have the same file name for different user
- Efficient searching
- Grouping capability

b) Acyclic-graph directories [5m]



Text book 10.3.6, slide 13.25

- Files/subdirectories have two different names (aliasing)
 - Only one actual file exists, so any changes made by one person are immediately visible to the other.
- How do we implement shared files and subdirectories?
 - Can duplicate the *inode* information about the shared file/subdirectory in each of the subdirectories that “point” to the shared structure.
 - If some information changes about the file it had to be updated in several places.
 - Have a new directory entry type:
 - **Link** – another name (pointer) to an existing file or subdirectory.

6. [10 MARKS] Answer the following questions:

a) In the Unix system, directory listing is shown as below:

```
-rw-r--r--      1 pbg staff      9423      Apr 19 2018  program
drwx-----      3 pbg staff      1024      Aug 19 10:31  mail/
```

(i) Name and define the directory protection. [3m]

Directory name mail/
owner only allows to read write execute

(ii) Name and define the file protection. [3m]

Program owner can read and write ,
group can read only,
universe can read only

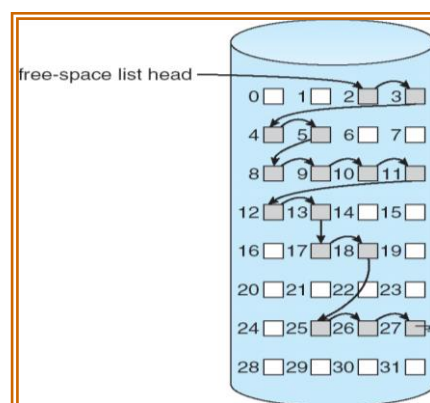
(iii) How to let all the users use the file without restriction? [3m]

chmod 777 program

7. [10 MARKS] Describe and show the example of free space management using:

a) Linked list [5m]

To link together all the free disk block. Keeping a pointer to the first free block in a special location on the disk and caching it in memory.



b) Grouping [5m]

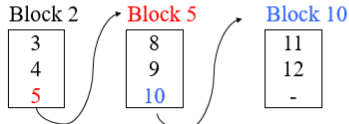
The first n-1 of these blocks are actually free. The last block contains the addresses of another n free blocks and so on. The addresses of a large number of free blocks can now be found quickly, unlike the linked-list

Grouping

- Store all the addresses of n free blocks in the first free block.

- Example:

– Free-block list: 2,3,4,5,8,9,10,11,12



Note block 2, 5 and 10 has become the index block. Therefore cannot be used for Data block

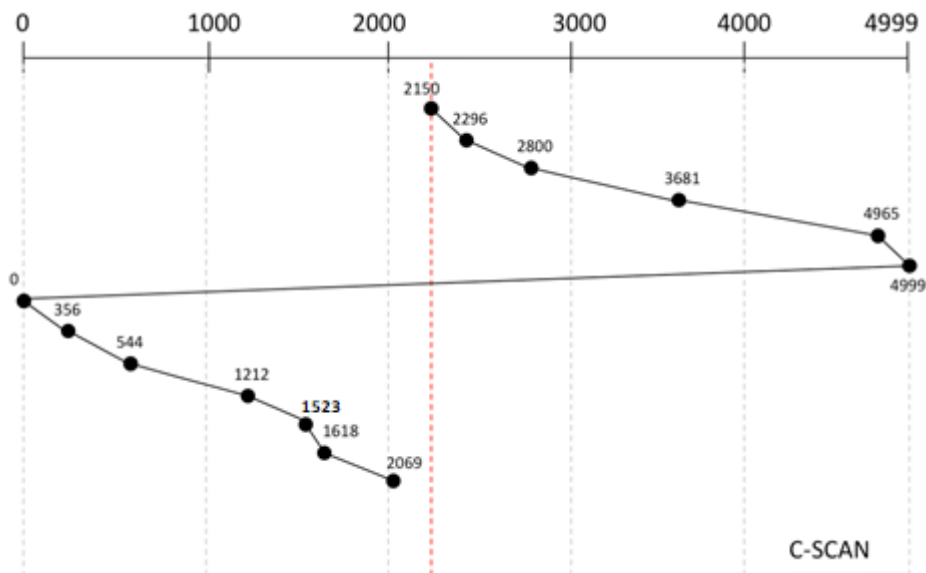
Assume each block can only stores 3 block addresses

8. [15 MARKS] Consider the following information:

- A disk drive has 5000 cylinders, numbered 0 – 4999.
- Using C-SCAN disk-scheduling algorithm to satisfy all the cylinder requests.
- The drive is currently serving a request at cylinder 2150.
- The queue of pending requests is: **2069, 1212, 2296, 2800, 544, 1618, 356, 1523, 4965, 3681**

Starting from the current head position, answer the following question.

a) Draw the whole disk arm movement to satisfy all the pending requests. [5m]



b) What is the total seek distance (in cylinder) that the disk arm had moved? [2m]

$$4999 - 2150 = 2849$$

$$4999 - 0 = 4999$$

$$2069 - 0 = 2069$$

Total distance = $2849 + 4999 + 2069 = 9917$ cylinders

c) Now, if C-LOOK algorithm is used to satisfy all the cylinder pending requests, compare the speed of both disk-scheduling algorithms based on the distance (in cylinder) that the disk arm had moved. [3m]

- C-LOOK uses total distance 780 less than S-SCAN
 - $(4999-4965) \times 2 = 34 \times 2 = 68$
 - $(356 \times 2) = 712$
 - total less = $68 + 712 = 780$
- therefore, C-LOOK faster than S-SCAN